13.4 Tratamento de erro, loading e estados vazios — status de entrega

Bloco 13.4 Tratamento de erro, loading e estados vazios · Sprint 13 — Administração, qualidade e segurança · Data: 2026-10-10 · Branch: `main` · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`)

Resumo

Validado: a rodada do bloco (`caffeinate -ims npm run validar` , 2026-10-10) passa nas sete camadas na primeira rodada completa. Números na seção 4.

O bloco não tem migration. Ele muda o que a pessoa vê quando algo dá errado, demora ou volta vazio:

- **Tela de erro e de "não encontrado".** O app não tinha nenhuma.
- **Skeleton nas listagens.** A tela deixa de ficar parada enquanto busca os dados.
- **Erro do banco em português.** Antes chegava à tela o texto do Postgres, em inglês.
- **"Limpar filtros" na listagem vazia por filtro.**
- **Toasts num padrão só.**

1. As cinco atividades do card

ATIVIDADE DO CLICKUP	ONDE FICOU
error.tsx e not-found.tsx globais e por rota	**Erro**: `src/app/global-error.tsx` (último recurso, quando nem o layout raiz renderiza) e `src/app/(app)/error.tsx` (dentro do shell, com "Tentar de novo", que refaz só a tela, e o código `digest` para avisar o administrador). **Não encontrado**: `src/app/not-found.tsx` (URL que não existe, fora do shell), `src/app/(app)/not-found.tsx` (genérico dentro do app) e um por detalhe, com a entidade e o caminho de volta: obra, cliente, orçamento, proposta, contrato, documento, FD, nota fiscal e acordo. O de cada `[id]` cobre também a edição abaixo dele. O componente é `NaoEncontrado`
loading.tsx com skeleton nas listagens	`ListagemSkeleton` (título, filtros e linhas da tabela no lugar em que vão aparecer) em 13 telas: as 10 listagens de cadastro e financeiro, mais `/logs` , `/execucao` e a visão geral. Seção 2
Mensagens de erro do Postgres traduzidas para linguagem de usuário	`mensagemDeErroBanco` e `mensagemDeErroHttp` , em `src/lib/erro-banco.ts` (4 testes). Seção 3

ATIVIDADE DO CLICKUP	ONDE FICOU
Revisar todos os EmptyState	O padrão já era bom e consistente: `EmptyState` com o convite a cadastrar quando a lista está vazia de verdade, e uma frase quando o vazio vem do filtro. Os 12 quadros de "nenhum resultado com esses filtros" eram cópias e não tinham saída. Agora são o componente `SemResultado`, com o mesmo texto e o atalho **Limpar filtros** (na execução, ele mantém a obra)
Revisar todos os toasts de sucesso e erro	149 toasts revisados. O de erro já seguia "Não foi possível X: motivo"; o problema era o motivo, que agora vem traduzido (seção 3). Seis fugiam do padrão: 3 "Falha no upload / no export / ao gerar" e 3 que mostravam o erro cru do Storage ou o JSON da resposta. Eles passaram ao padrão. Nos de sucesso, 5 "criado com sucesso" viraram "criado", como os outros

2. O skeleton e os redirects

`loading.tsx` envolve a página **e as rotas filhas** do segmento. Posto em `/propostas`, ele envolveria também `/propostas/nova` e `/propostas/[id]/editar`. O `redirect()` de perfil dessas páginas sairia por streaming, como `<meta http-equiv="refresh">` com status 200, em vez de 307. A varredura de URL direta do 13.3 espera 307 de quem não pode, e com razão.

Por isso, nas 10 listagens que têm rotas filhas, a `page.tsx` foi para um grupo `(lista)`, que não muda a URL, e o `loading.tsx` fica só ali: `propostas/(lista)/page.tsx` e `propostas/(lista)/loading.tsx`. O build confirma as mesmas 65 rotas. `/logs`, `/execucao` e a visão geral não têm filhas, e o `loading.tsx` fica direto no segmento.

Uma consequência, assumida: nas listagens, a página além do fim (`?page=99`) redireciona para a primeira. Agora o skeleton já foi enviado quando a page decide redirecionar, então o Next manda o redirect no próprio HTML (`<meta id="__next-page-redirect" http-equiv="refresh">`, status 200), e não como 307. Para quem usa não muda nada: a tela volta à primeira página com os filtros, conferido no navegador. Os 11 testes desse caso mudaram de "espera 307" para conferir o destino exato do redirect, que antes nem era conferido, e a presença do skeleton.

3. Erro do banco em linguagem de usuário

Antes, 33 retornos de Server Action e as 7 exportações devolviam `error.message` cru. Na tela aparecia, por exemplo, "update or delete on table "obras" violates foreign key constraint". Agora:

- **`mensagemDeErroBanco`** traduz pela família do erro, pelo código do Postgres ou pelo texto:

FAMÍLIA	FRASE NA TELA
permissão	"Você não tem permissão para fazer isso."
duplicado	"Já existe um registro com esses dados."
registro ligado	"Não dá para excluir: há outros registros ligados a este."

FAMÍLIA	FRASE NA TELA
campo obrigatório	"Falta preencher um campo obrigatório."
valor fora do permitido	"Algum valor está fora do permitido. Confira os campos."

O mesmo vale para formato inválido, texto longo demais, conflito de edição, tempo esgotado, sessão vencida, falta de conexão e os erros do Storage (arquivo repetido, grande demais, tipo não aceito).

  - Frase nossa (exceção de trigger, em português) passa como está.
  - Inglês de sistema que nenhuma regra pegou vira "Não foi possível concluir. Tente de novo; se continuar, avise o administrador.", e o texto cru não aparece.

- **Tradutores de domínio** (`mensagemDeErroNf`, `mensagemDeErroContrato` e os outros seis): conhecem as constraints deles pelo nome e, no que não conhecem, agora caem no genérico em vez de devolver o cru.
- **`mensagemDeErroHttp`**: usada pelos botões de exportação e de relatório. 401 vira sessão, 403 vira perfil, 400 mostra a frase da rota ("Escolha uma obra") e 500 passa pelo tradutor do banco. Antes, o toast mostrava `{"error":"Forbidden"}`.
- **Auditoria:** a mensagem crua continua lá (13.2, etapa B). A tela simplifica, e o log guarda o detalhe.
- **Rotas de ingestão:** ficam de fora de propósito, porque quem lê é o n8n.

4. Validação

Rodada do bloco (`caffeinate -ims npm run validar`, 2026-10-10):

CAMADA	RESULTADO	DO 13.4
sobras	nenhuma em gc-dev	—
estático	**limpo** (`tsc` + `next lint`)	—
unitário	**397 casos**	**4 casos** em `erro-banco.test.ts`: as famílias do Postgres, sessão, conexão e Storage, frase nossa contra inglês de sistema, e a resposta HTTP das exportações
build	**65 rotas**, iguais à baseline	nenhuma rota nova: o grupo `(lista)` não muda URL
runtime	**319/319 rotas** e as **342 células** da varredura de URL direta, sem rota furada	**21 rotas novas**: o skeleton nas 13 telas, o "não encontrado" de proposta (detalhe e edição), obra, orçamento, FD, documento, cliente e o global, com o texto. Mais o "Limpar filtros" em 6 buscas vazias, o texto nos 3 detalhes 404 que já existiam e os 11 redirects de página além do fim pelo HTML
dados	**60/60 checagens**	nada novo: o bloco não tem query

CAMADA	RESULTADO	DO 13.4
escrita	**506/506 passos**	**1 passo**: o número de orçamento repetido bate no índice único do banco e volta como "Já existe um registro com esses dados.", sem o texto do Postgres
navegador	**260/260 passos**	**4 checagens**: a proposta inexistente mostra "Proposta não encontrada" dentro do shell, e o botão volta à listagem; a busca vazia em contratos oferece "Limpar filtros", e o clique volta à listagem com as linhas. Screenshots `57-nao-encontrado.png` e `57-limpar-filtros.png`. Um passo antigo (o filtro de `/logs`) passou a esperar o conteúdo, porque a URL agora muda antes, enquanto a tela é o skeleton

Antes da rodada: o runtime isolado mostrou exatamente os 11 redirects de página previstos na seção 2, e só eles. O navegador isolado parou uma vez no passo de `/logs` (o skeleton) e passou com 260/260 depois do ajuste.

5. Pendências

- **Conferência visual do skeleton e das telas de erro:** a cargo do Breno. A camada navegador prova que as telas renderizam e que o texto está lá, não a aparência.
- **Tela de erro (`error.tsx`) só se exercita com uma falha de verdade no servidor.** O plano não quebra uma tela de propósito; o arquivo está lá, mas o caminho não roda na validação.
- `global-error.tsx` nunca roda em `next dev`; o Next mostra o overlay de erro no lugar.